

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ ЖИВОТНОГО МИРА
НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное
учреждение Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

День 20.

**«Интегрированная система безопасности предприятия (физическая
+ ИТ)»**

Выполнил: Рахманов А. И.

Студент 24-ИС.

р.п. Красные Баки 2026г.

Оглавление

Введение	3
Цель	3
Вариант	3
Структура.....	3
Глава 1. Система контроля доступа (СКУД)	4
Управление пропусками	4
Глава 2. Мониторинг киберугроз.....	5
SIEM-система: сбор и корреляция событий безопасности	5
Глава 3. Внедрите обнаружение аномалий	10
Распознавание лиц	13
Глава 4. Управление инцидентами	15
Код программы не проходящей тесты	15
Код тестов и уведомлений.....	16
Запуск тестов.....	20
Вывод	21

Введение

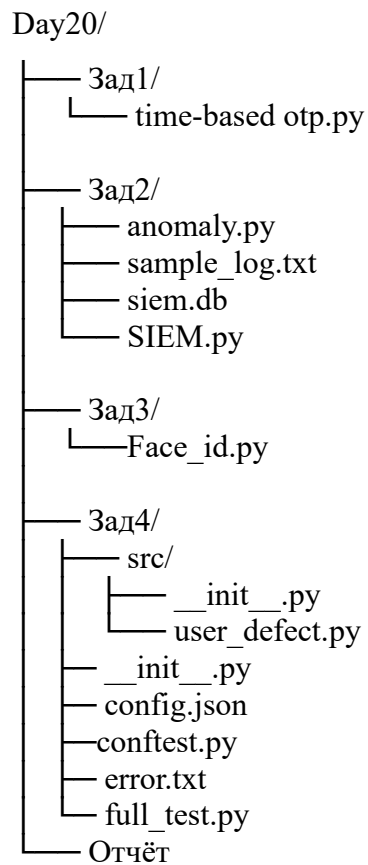
Цель

Интегрировать систему безопасности (СКУД, Киберугрозы, Биометрия, Инциденты) и модули уведомления для контроля качества в систему безопасности предприятия.

Вариант

Вариант 10. Безопасность и контроль (СКУД, Киберугрозы, Биометрия, Инциденты)

Структура



Глава 1. Система контроля доступа (СКУД)

Управление пропусками

Данная программа распознаёт пользователя из выданной таблицы и даёт разрешение на пропуск, беря информацию о состоянии сотрудника(отпуска) из базы данных.

```
employers = [
    {'emp_id': '101', 'name': 'Иван', 'on_vacation': False},
    {'emp_id': '102', 'name': 'Иван', 'on_vacation': True},
    {'emp_id': '103', 'name': 'Иван', 'on_vacation': False}
]
turnstile = {'turnstile_id': '1'}
def turnstile_access(emp_id, turnstile_id):
    employee = None
    for emp in employers:
        if emp['emp_id'] == str(emp_id):
            employee = emp
            break
    if employee is None:
        print('no accept, user out of database')
        return False
    if employee['on_vacation']:
        print(f'no accept {employee["name"]} on vacation')
        return False
    open_turn()
    return True
def open_turn():
    print('accept access')
    return True
turnstile_access(102, 1)
turnstile_access(101, 1)
```

Приложение 1. Программа анти-перехватного алгоритма

Глава 2. Мониторинг киберугроз

SIEM-система: сбор и корреляция событий безопасности

Данная программа является SIEM-системой сбора логов в sample_log.txt

```
import sys
import re
import sqlite3
import time
from datetime import datetime
from collections import defaultdict

from PyQt6.QtWidgets import (
    QApplication, QWidget, QVBoxLayout, QPushButton,
    QTextEdit, QLabel, QLineEdit, QHBoxLayout, QTabWidget
)
from PyQt6.QtCore import QThread, pyqtSignal

# CONFIG
LOG_FILE = "sample_logs.txt"
FAILED_THRESHOLD = 5

# DATABASE
def init_db():
    conn = sqlite3.connect("siem.db")
    c = conn.cursor()
    c.execute("""
        CREATE TABLE IF NOT EXISTS alerts (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            ip TEXT,
            type TEXT,
            attempts INTEGER,
            severity TEXT,
            timestamp TEXT
        )
    """)
    conn.commit()
    conn.close()

def insert_alert(alert):
    conn = sqlite3.connect("siem.db")
    c = conn.cursor()
    c.execute("""
        INSERT INTO alerts (ip, type, attempts, severity, timestamp)
        VALUES (?, ?, ?, ?, ?)
    """, (alert["ip"], alert["type"], alert["attempts"],
    alert["severity"], alert["time"])))
```

```

conn.commit()
conn.close()

def fetch_alerts(filter_ip=None):
    conn = sqlite3.connect("siem.db")
    c = conn.cursor()

    if filter_ip:
        c.execute("SELECT * FROM alerts WHERE ip LIKE ?", ('%' +
filter_ip + '%',))
    else:
        c.execute("SELECT * FROM alerts ORDER BY id DESC")

    rows = c.fetchall()
    conn.close()
    return rows

# DETECTION ENGINE
FAILED_PATTERN = re.compile(r"Failed password for .* from
(\d+\.\d+\.\d+\.\d+)")

def analyze_logs():
    failed = defaultdict(int)

    try:
        with open(LOG_FILE, "r") as f:
            for line in f:
                match = FAILED_PATTERN.search(line)
                if match:
                    ip = match.group(1)
                    failed[ip] += 1
    except FileNotFoundError:
        return []

    alerts = []

    for ip, count in failed.items():
        if count >= FAILED_THRESHOLD:
            alerts.append({
                "ip": ip,
                "type": "Brute Force",
                "attempts": count,
                "severity": "High",
                "time": datetime.now().strftime("%Y-%m-%d %H:%M:%S")
            })

    return alerts

```

```

# MONITOR THREAD
class MonitorThread(QThread):
    alert_signal = pyqtSignal(dict)

    def run(self):
        seen = set() # avoid duplicate spam

        while True:
            alerts = analyze_logs()

            for alert in alerts:
                key = (alert["ip"], alert["attempts"])
                if key not in seen:
                    seen.add(key)
                    insert_alert(alert)
                    self.alert_signal.emit(alert)

            time.sleep(5)

# GUI
class SIEMApp(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("🔒 SIEM Pro Dashboard")
        self.setGeometry(200, 200, 900, 600)
        self.setStyleSheet(self.style())

        self.tabs = QTabWidget()
        self.dashboard_tab()
        self.alerts_tab()

        layout = QVBoxLayout()
        layout.addWidget(self.tabs)
        self.setLayout(layout)

        self.start_monitoring()

# ----- Dashboard ----- #
def dashboard_tab(self):
    tab = QWidget()
    layout = QVBoxLayout()

    self.stats_label = QLabel("Total Alerts: 0")
    layout.addWidget(self.stats_label)

    refresh_btn = QPushButton("Refresh Stats")
    refresh_btn.clicked.connect(self.update_stats)
    layout.addWidget(refresh_btn)

    tab.setLayout(layout)

```

```

        self.tabs.addTab(tab, "Dashboard")

# ----- Alerts ----- #
def alerts_tab(self):
    tab = QWidget()
    layout = QVBoxLayout()

    search_layout = QHBoxLayout()

    self.search_input = QLineEdit()
    self.search_input.setPlaceholderText("Search by IP...")

    search_btn = QPushButton("Search")
    search_btn.clicked.connect(self.load_alerts)

    search_layout.addWidget(self.search_input)
    search_layout.addWidget(search_btn)

    layout.addLayout(search_layout)

    self.output = QTextEdit()
    self.output.setReadOnly(True)
    layout.addWidget(self.output)

    tab.setLayout(layout)
    self.tabs.addTab(tab, "Alerts")

# ----- Monitoring ----- #
def start_monitoring(self):
    self.thread = MonitorThread()
    self.thread.alert_signal.connect(self.display_alert)
    self.thread.start()

# ----- UI Updates ----- #
def display_alert(self, alert):
    self.output.append(
        f"[{alert['severity']}] {alert['type']} → {alert['ip']} "
        f"({alert['attempts']} attempts)"
    )
    self.update_stats()

def load_alerts(self):
    self.output.clear()
    rows = fetch_alerts(self.search_input.text())

    for row in rows:
        _, ip, typ, attempts, severity, timestamp = row
        self.output.append(
            f"[{severity}] {typ} → {ip} ({attempts}) @ {timestamp}"
        )

```



```

def update_stats(self):
    rows = fetch_alerts()
    self.stats_label.setText(f"Total Alerts: {len(rows)}")

def style(self):
    return """
QWidget { background:#0f172a; color:#e2e8f0; font-family:Segoe
UI; }
QPushButton { background:#2563eb; padding:6px; border-
radius:6px; }
QPushButton:hover { background:#1d4ed8; }
QLineEdit { background:#1e293b; padding:6px; border-radius:6px;
}

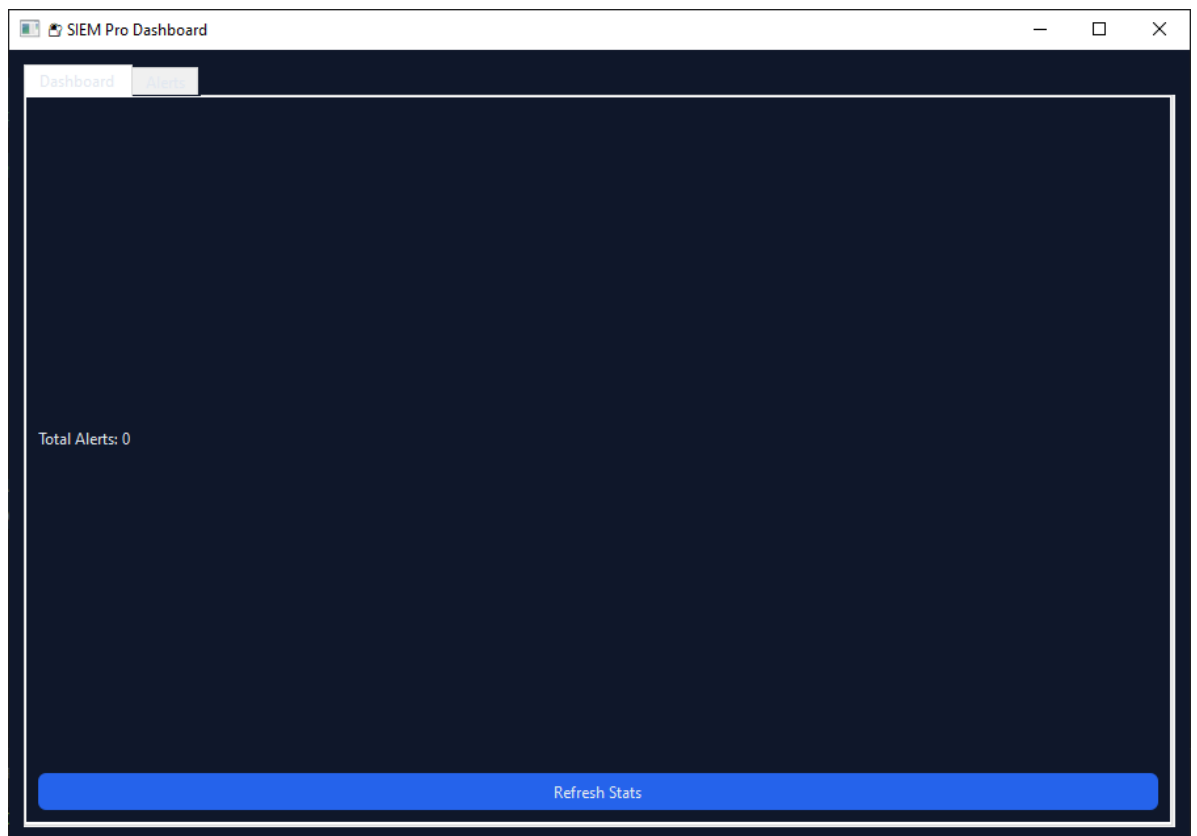
QTextEdit { background:#020617; border-radius:6px; }
"""

# RUN
if __name__ == "__main__":
    init_db()

    app = QApplication(sys.argv)
    window = SIEMApp()
    window.show()
    sys.exit(app.exec())

```

Приложение 2. SIEM-система.



Приложение 3. Работа SIEM-системы

Глава 3. Внедрите обнаружение аномалий

Данная программа является системой обнаружения аномальных значений при работе.

```
import os
import numpy as np
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
from matplotlib.colors import Normalize

def gen_normal_distribution(mu, sigma, size, range=(0, 1), max_val=1):
    bins = np.linspace(*range, size)
    result = 1 / (sigma * np.sqrt(2*np.pi)) * np.exp(-(bins - mu)**2 /
(2*sigma**2))
    cur_max_val = result.max()
    k = max_val / cur_max_val
    result *= k
    return result

dist = gen_normal_distribution(0.3, 0.05, 256, max_val=1)
print(f"Max value: {dist.max()}")

plt.figure(figsize=(10, 4))
plt.plot(np.linspace(0, 1, 256), dist)
plt.title("Sample Normal Distribution")
plt.show()

in_distribution_size = 2000
out_distribution_size = 200
val_size = 100
sample_size = 256

random_generator = np.random.RandomState(seed=42)

def generate_in_samples(size, sample_size):
    in_samples = np.zeros((size, sample_size))

    in_mus = random_generator.uniform(0.1, 0.9, size)
    in_sigmas = random_generator.uniform(0.05, 0.5, size)
    for i in range(size):
        in_samples[i] = gen_normal_distribution(in_mus[i], in_sigmas[i],
sample_size, max_val=1)
    return in_samples

def generate_out_samples(size, sample_size, anomaly_height=0.12):

    out_samples = generate_in_samples(size, sample_size)
```

```

out_additional_mus = random_generator.uniform(0.1, 0.9, size)
out_additional_sigmas = random_generator.uniform(0.01, 0.05, size)

for i in range(size):

    anomaly = gen_normal_distribution(
        out_additional_mus[i],
        out_additional_sigmas[i],
        sample_size,
        max_val=anomaly_height
    )
    out_samples[i] += anomaly
return out_samples

in_samples = generate_in_samples(in_distribution_size, sample_size)
out_samples = generate_out_samples(out_distribution_size, sample_size)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

ax1.plot(np.linspace(0, 1, sample_size), in_samples[42])
ax1.set_title("In-Distribution Sample (Single Peak)")
ax1.set_xlabel("Value")
ax1.set_ylabel("Density")

ax2.plot(np.linspace(0, 1, sample_size), out_samples[42])
ax2.set_title("Out-of-Distribution Sample (Two Peaks)")
ax2.set_xlabel("Value")
ax2.set_ylabel("Density")

plt.tight_layout()
plt.show()

X = np.vstack([in_samples, out_samples])
y = np.hstack([np.ones(in_distribution_size),
np.zeros(out_distribution_size)])

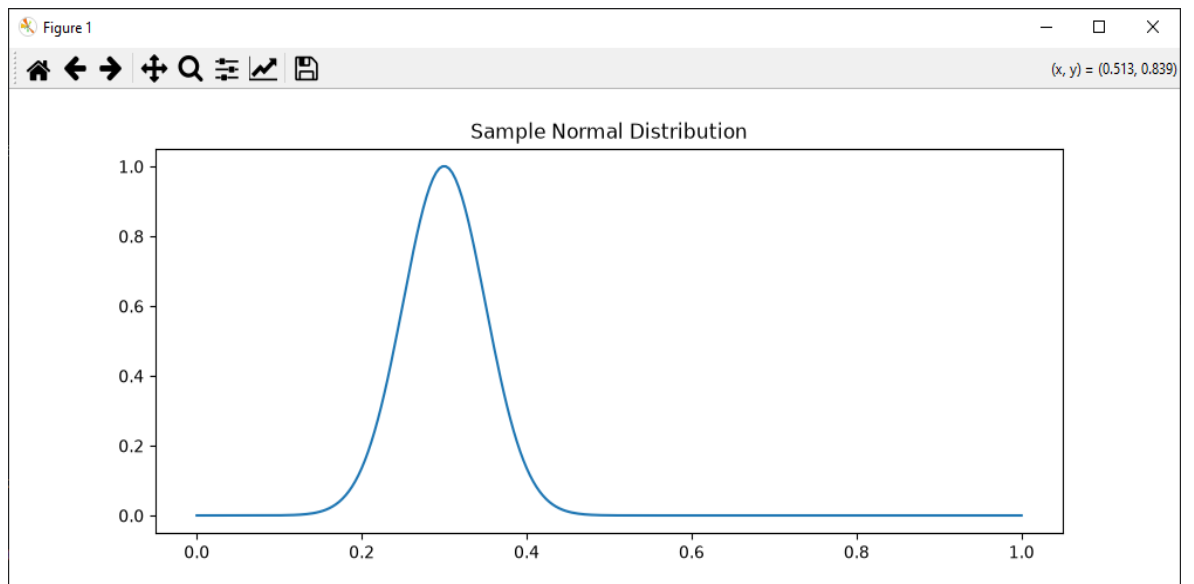
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

X_val, X_test, y_val, y_test = train_test_split(
    X_test, y_test, test_size=0.5, random_state=42, stratify=y_test
)

print(f"Train set size: {len(X_train)} (In: {sum(y_train)}, Out: {len(y_train)-sum(y_train)})")
print(f"Validation set size: {len(X_val)} (In: {sum(y_val)}, Out: {len(y_val)-sum(y_val)})")
print(f"Test set size: {len(X_test)} (In: {sum(y_test)}, Out: {len(y_test)-sum(y_test)})")

```

Приложение 4. Система обнаружения аномалий.



Приложение 5. Работа COA.

Распознавание лиц

Данная программа является программой распознавания лиц и сверения её с базой данных людей в розыске.

```
import cv2
import numpy as np
#uncomment if u wanna check
#from Facedb import Face_db

#db = Face_db(path='wanted_db', module= 'deepface')

video = cv2.VideoCapture(0)

if not video.isOpened():
    print("Error: Could not open camera")
    exit()

facedetect =
cv2.CascadeClassifier('data/haarcascade_frontalface_default.xml')

if facedetect.empty():
    print("Error: Could not load cascade classifier")
    exit()

faces_data = []
i = 0

while True:
    ret, frame = video.read()

    if not ret:
        print("Warning: Failed to capture frame, retrying...")
        continue # Skip this iteration and try again

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = facedetect.detectMultiScale(gray, 1.3, 5)

    for (x, y, w, h) in faces:
        crop_img = frame[y:y+h, x:x+w, :]
        resized_img = cv2.resize(crop_img, (50, 50))

        if len(faces_data) <= 100 and i % 10 == 0:
            faces_data.append(resized_img)
            i = i + 1

        cv2.putText(frame, str(len(faces_data)), (50, 50),
cv2.FONT_HERSHEY_COMPLEX, 1, (50, 50, 255), 1)
```

```

        cv2.rectangle(frame, (x, y), (x+w, y+h), (50, 50, 255), 1)

cv2.imshow("Frame", frame)
k = cv2.waitKey(1)
if k == ord('q'):
    break

# uncomment if u wanna check
# result = db.recognize(img=face, include=['name', 'distance'])
# if result:
#     name = result['name']
#     distance = result['distance'] # Fixed spelling from
'destance'
#     if distance < 0.4:
#         print('some bad person captured')
#     else:
#         print('idk')

video.release()
cv2.destroyAllWindows()

if faces_data:
    print(f"Collected {len(faces_data)} face samples")

    faces_data = np.array(faces_data)
    np.save('faces_data.npy', faces_data)

```

Приложение 6. Программа распознавания лиц.

Глава 4. Управление инцидентами

Данная программа является системой оповещения при неудачном прохождении тестов с подключением программы user_defect.py.

Код программы не проходящей тесты

```
# src/user.py
"""
Модуль для управления пользователями в системе бронирования
"""
import re
from typing import Optional

def validate_email(email: str) -> bool:
    """Проверить корректность email"""
    # ОШИБКА: слишком простая валидация, пропускает невалидные email
    if '@' not in email:
        return False
    # ОШИБКА: не проверяет наличие домена после точки
    return True

def validate_password(password: str) -> bool:
    """Проверить надежность пароля"""
    # Требования: минимум 8 символов, есть цифра и спецсимвол
    if len(password) < 8:
        return False
    # ОШИБКА: проверяет только наличие цифры, но не спецсимволов
    if not any(c.isdigit() for c in password):
        return False
    # ОШИБКА: всегда возвращает True, даже если нет спецсимволов
    return True

def validate_age(age: int) -> bool:
    """Проверить возраст (должен быть 18-120)"""
    # ОШИБКА: пропускает отрицательные значения
    if age < 18:
        return False
    # ОШИБКА: не проверяет верхнюю границу
    return True

def create_user(email: str, password: str, age: int) -> dict:
    """Создать пользователя с валидацией"""
    # ОШИБКА: не проверяет age на тип (может быть None)
    if not validate_email(email):
        raise ValueError("Invalid email")
    if not validate_password(password):
        raise ValueError("Invalid password")
    if not validate_age(age):
        raise ValueError("Invalid age")
```

```

    return {
        "email": email,
        "password": "****", # хэш в реальной системе
        "age": age,
        "active": True
    }

def update_user_age(user: dict, new_age: int) -> dict:
    """Обновить возраст пользователя"""
    # ОШИБКА: не проверяет, что пользователь существует
    # ОШИБКА: не проверяет валидность нового возраста
    user["age"] = new_age
    return user

```

Приложение 7. Проблемная программа.

Код тестов и уведомлений

```

import sys
import os
import pytest
import requests
import json
from datetime import datetime

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from src.user_defect import (
    validate_email,
    validate_password,
    validate_age,
    create_user,
    update_user_age
)

def load_config():
    config_path = os.path.join(os.path.dirname(__file__), 'config.json')
    try:
        with open(config_path, 'r', encoding='utf-8') as f:
            return json.load(f)
    except FileNotFoundError:
        print("Файл config.json не найден. Telegram уведомления отключены.")
        return {}
    except json.JSONDecodeError:
        print("Ошибка в формате config.json. Telegram уведомления отключены.")
        return {}

config = load_config()
TELEGRAM_BOT_TOKEN = config.get('telegram_bot_token', '')

```



```

TELEGRAM_CHAT_ID = config.get('telegram_chat_id', '')

def send_telegram_notification(test_name, error_message,
test_file="full_test.py"):
    if not TELEGRAM_BOT_TOKEN or not TELEGRAM_CHAT_ID:
        print("Telegram не настроен. Пропускаем уведомление.")
        return

    if len(error_message) > 500:
        error_message = error_message[:500] + "..."

    message = f"""ОШИБКА ТЕСТА

Файл: {test_file}
Тест: {test_name}
Ошибка:
{error_message}
Время: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}
"""

    url =
f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/sendMessage"
    payload = {
        "chat_id": TELEGRAM_CHAT_ID,
        "text": message
    }

    try:
        response = requests.post(url, json=payload, timeout=5)
        response.raise_for_status()
        print("Уведомление в Telegram отправлено")
    except Exception as e:
        print(f"Ошибка отправки в Telegram: {e}")

@pytest.fixture(autouse=True)
def notify_on_failure(request):
    yield
    if hasattr(request.node, 'rep_call') and
request.node.rep_call.failed:
        test_name = request.node.name
        error_message = str(request.node.rep_call.longrepr)
        send_telegram_notification(test_name, error_message)

def test_validate_email_valid():
    assert validate_email("user@example.com") is True
    assert validate_email("test@mail.ru") is True

def test_validate_email_no_at():
    assert validate_email("userexample.com") is False

def test_validate_email_empty():

```

```

    assert validate_email("") is False

@pytest.mark.parametrize("email, expected", [
    ("user+tag@example.com", True),
    ("user.name@example.com", True),
    ("user_name@example.com", True),
    ("user@sub.domain.com", True),
    ("user@example.co.uk", True),
    ("user@example", False),
    ("user@.com", False),
    ("user@example.c", False),
    ("user@exa_mple.com", False),
])

def test_validate_email_parametrized(email, expected):
    assert validate_email(email) == expected

def test_validate_password_valid():
    assert validate_password("Passw0rd!") is True
    assert validate_password("P@ssw0rd") is True

def test_validate_password_too_short():
    assert validate_password("Pass1!") is False

def test_validate_password_no_digit():
    assert validate_password("Password!") is False

def test_validate_password_no_special():
    assert validate_password("Password1") is False

@pytest.mark.parametrize("password, expected", [
    ("Passw0rd!", True),
    ("Pass123!", True),
    ("Passw0rd", False),
    ("P@ssw0rd", True),
    ("Passwo1!", True),
    ("12345678!", False),
    ("Passw0rd@", True),
    ("Abc123!!", True),
    ("short1!", False),
    ("NoSpecial1", False),
    ("NoDigit!", False),
])

def test_validate_password_parametrized(password, expected):
    assert validate_password(password) == expected

def test_validate_age_valid():
    assert validate_age(25) is True
    assert validate_age(18) is True
    assert validate_age(120) is True

def test_validate_age_too_young():

```

```

    assert validate_age(17) is False

def test_validate_age_too_old():
    assert validate_age(121) is False
    assert validate_age(150) is False

def test_validate_age_negative():
    assert validate_age(-5) is False

@pytest.mark.parametrize("age, expected", [
    (18, True),
    (120, True),
    (25, True),
    (17, False),
    (121, False),
    (0, False),
    (-5, False),
    (150, False),
])

def test_validate_age_parametrized(age, expected):
    assert validate_age(age) == expected

def test_validate_age_non_integer():
    assert validate_age(25.5) is False
    assert validate_age("25") is False
    assert validate_age(None) is False

def test_create_user_valid():
    user = create_user("test@example.com", "Passw0rd!", 25)
    assert user["email"] == "test@example.com"
    assert user["age"] == 25
    assert user["active"] is True

def test_create_user_invalid_email():
    with pytest.raises(ValueError, match="Invalid email"):
        create_user("invalid", "Passw0rd!", 25)

def test_create_user_invalid_password():
    with pytest.raises(ValueError, match="Invalid password"):
        create_user("test@example.com", "password", 25)

def test_create_user_invalid_age():
    with pytest.raises(ValueError, match="Invalid age"):
        create_user("test@example.com", "Passw0rd!", 15)

def test_create_user_age_121():
    with pytest.raises(ValueError, match="Invalid age"):
        create_user("test@example.com", "Passw0rd!", 121)

def test_create_user_invalid_age_type():
    with pytest.raises(ValueError, match="Age must be integer"):

```

```

        create_user("test@example.com", "Passw0rd!", 25.5)

def test_update_user_age_valid():
    user = create_user("test@example.com", "Passw0rd!", 25)
    updated = update_user_age(user, 30)
    assert updated["age"] == 30

def test_update_user_age_invalid():
    user = create_user("test@example.com", "Passw0rd!", 25)
    with pytest.raises(ValueError, match="Invalid age"):
        update_user_age(user, 15)

def test_update_user_age_empty_user():
    with pytest.raises(ValueError, match="User not found"):
        update_user_age(None, 30)

```

Приложение 8. Код тестов.

Запуск тестов

- `cd Y:\24 ИСиП 2025\УП02\Группа 2\Рахманов А\day20`
- `pytest Зад4/full_test.py -v`

Вывод

За работу проделанную на данном занятии я смог написать 4 модуля системы для интегрирования в систему безопасности организации. Были написаны модули: система контроля доступа, мониторинг киберугроз, обнаружения аномалий, система распознавания лиц и сверения с базой данных розыска и система тестирования с отправлением уведомлений на почту. За работу были узнаны новые модули и библиотеки в python, закрепились знания и умения писать код. Данные знания пригодятся для дальнейшего обучения и работы по специальности.